

■ 2.1.3 Four Ways to Write Expressions

Internally, *Mathematica* represents a sum of two objects as `Plus[x, y]`. You can, however, enter this expression in the much more convenient form `x + y`. *Mathematica* also accepts some alternative ways of writing standard expressions like `f[x, y, ...]`.

<code>f[x, y]</code>	standard form for <code>f[x, y]</code>
<code>f @ x</code>	prefix form for <code>f[x]</code>
<code>x // f</code>	postfix form for <code>f[x]</code>
<code>x ~ f ~ y</code>	infix form for <code>f[x, y]</code>

Four ways to write expressions in *Mathematica*.

This “postfix form” is exactly equivalent to `f[x + y]`.

```
In[1]:= x + y //f
Out[1]= f[x + y]
```

You will often want to add functions like `N` as “afterthoughts”, and give them in postfix form.

```
In[2]:= 3^(1/4) + 1 //N
Out[2]= 2.31607
```

It is sometimes easier to understand what a function is doing when you write it in infix form.

```
In[3]:= {a, b, c} ~Join~ {d, e}
Out[3]= {a, b, c, d, e}
```

You should notice that `//` has very low “precedence”. If you put `//f` at the end of any expression containing arithmetic or logical operators, the `f` is applied to the *whole expression*. So, for example, `x+y //f` means `f[x+y]`, not `x+f[y]`.

The prefix form `@` works differently. `f @ x + y` is equivalent to `f[x] + y`, not `f[x + y]`. You can write `f[x + y]` in prefix form as `f @ (x + y)`.